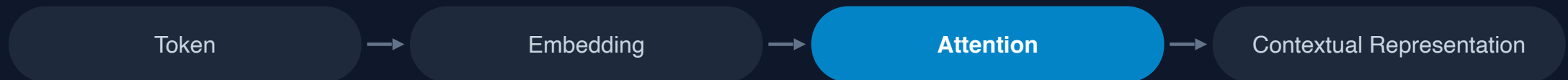


# Attention

## From Token Embeddings to Contextual Representations

*If every token starts with its own embedding, how does the model understand that the meaning of a token depends on the other tokens around it?*



# What You Will Learn Today

1

## Why Attention

Why token embeddings alone can't capture meaning that depends on context.

2

## Query, Key, Value

How a token asks for, offers, and delivers contextual information.

3

## Scores & Softmax

Turning similarity into normalized weights, then a weighted sum of Values.

4

## Self & Causal Attention

Full-context attention vs. left-to-right masked attention (GPT-style).

5

## Multi-Head & Cross-Attention

Parallel relationships in one sequence; linking two sequences together.

6

## Into the Transformer

How attention becomes the core building block of the Transformer block.

# From Embeddings to Attention

## LECTURES 4–7

### Embeddings & RAG

Tokens become fixed vectors from an embedding matrix. Similarity is measured in that vector space.



## LECTURE 8 · TODAY

### Attention

Tokens exchange information with each other to build context-aware representations.



## LECTURE 9 · NEXT

### The Transformer

Attention + feed-forward layers, stacked with residuals and normalization.

*Same question carries through the whole course: how does a model turn raw tokens into something that understands context?*

# Does an Embedding Know Its Context?

**The bank approved the loan.**

After tokenization, each token gets its own embedding:



The embedding of **bank** comes only from the embedding matrix — it does not directly encode that this sentence is about a financial institution.

## ASK THE ROOM

*“Does the embedding of bank know that loan is related to its meaning?”*

**No.**

We need a mechanism that lets bank look at its surrounding tokens and construct a contextual representation. That mechanism is Attention.

# Same Token, Different Meaning

## SENTENCE 1

**I deposited money in the bank.**

bank should attend to:

**money**

**deposited**

→ **Financial context**

## SENTENCE 2

**We sat on the bank of the river.**

bank should attend to:

**river**

**sat**

→ **Geographical context**

*Same initial embedding for bank. Different meaning. We need a mechanism to build that context — that mechanism is Attention.*

# What Is Attention?

**Attention is a mechanism that allows one token to selectively gather information from other tokens.**

The bank approved the loan

*Illustrative attention weights for bank*



*The key word is selectively — attention does not treat every token equally.*

# What Is bank Even Looking For?

*bank essentially needs to ask: “Which other tokens are important to me?”*

**I deposited money in the bank.**

Relevant: **money, deposited**

→ Financial relevance

**We walked beside the bank of the river.**

Relevant: **river, walked**

→ Geographical relevance

**So attention needs three things:**

**Query**

What am I looking for?

**Key**

What kind of information do I contain?

**Value**

What information can I provide?

# Introducing Query, Key, Value

For every token, we create three vectors from its embedding:

$$Q = E \cdot W_Q \quad K = E \cdot W_K \quad V = E \cdot W_V$$



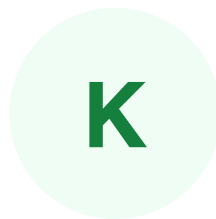
*$W_Q$ ,  $W_K$ ,  $W_V$  are learned weight matrices — every token in the sequence goes through the same projection.*

# What Do Q, K, and V Represent?



## Query

Represents what this token is looking for — the question it asks of the rest of the sequence.



## Key

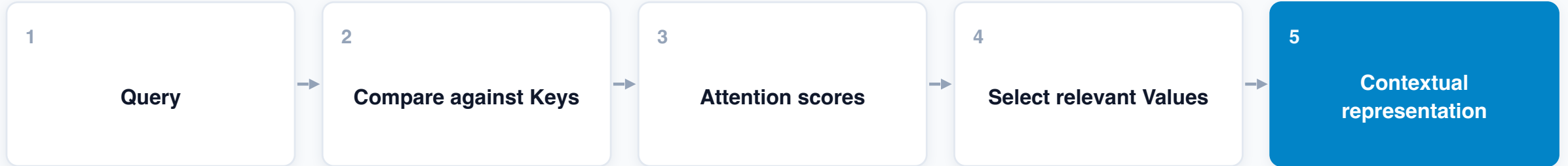
Represents what this token offers, or what it can be matched against by other tokens' queries.



## Value

Represents the actual information that gets passed forward once a match is found.

# Attention as a Search Engine



bank is searching a database. Every other token has a Key describing what it offers:

|                 |                        |                                             |
|-----------------|------------------------|---------------------------------------------|
| <b>money</b>    | → financial-related    | $Q_{\text{bank}} \cdot K_{\text{money}}$    |
| <b>river</b>    | → geographical-related | $Q_{\text{bank}} \cdot K_{\text{river}}$    |
| <b>approved</b> | → action-related       | $Q_{\text{bank}} \cdot K_{\text{approved}}$ |

*Large score = more relevant. Retrieve the corresponding Values.*

# The Attention Equation

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

## 1 Calculate similarity

$$Q K^T$$

The dot product of Query and Key vectors gives raw attention scores.

## 2 Scale

$$\div \sqrt{d_k}$$

Large dimensions make dot products large, which makes softmax too sharp. Scaling keeps gradients stable.

# Softmax & the Weighted Sum

3

## Softmax

**$\text{softmax}(\text{scores})$**

Converts scores into weights that all sum to 1 — a normalized probability-like distribution over tokens.

$$0.05 + 0.10 + 0.60 + 0.25 = 1.00$$

4

## Weighted sum of Values

**$\text{Weights} \times V$**

Each token's Value is scaled by its attention weight and summed — producing the new contextual representation.

## A Tiny Numerical Example

**Q\_bank = [1, 1]**

K\_money = [1, 1]      K\_river = [-1, 1]      K\_approved = [1, 0]

| Comparison          | Dot Product    | Score |
|---------------------|----------------|-------|
| Q_bank · K_money    | $1(1) + 1(1)$  | 2     |
| Q_bank · K_approved | $1(1) + 1(0)$  | 1     |
| Q_bank · K_river    | $1(-1) + 1(1)$ | 0     |

### READING THE SCORES

money scores highest (2) — bank is most relevant to money in this context.

*Next: turn these raw scores into normalized weights with softmax, then compute the output.*

## From Scores to a New Representation

Softmax turns  $[2, 1, 0]$  into normalized weights:



$V_{\text{money}} = [2, 0]$      $V_{\text{approved}} = [1, 1]$      $V_{\text{river}} = [0, 2]$

$$\text{Output\_bank} = 0.67[2,0] + 0.24[1,1] + 0.09[0,2] \approx [1.58, 0.42]$$

*The new bank representation is no longer just its own embedding — it now contains information gathered from its context.*

# Self-Attention

Self-attention is attention where Q, K, and V are all generated from the same sequence.



*Every token can potentially attend to every other token*

$$Q = X W_Q \quad K = X W_K \quad V = X W_V$$

$$\text{Attention}(X) = \text{softmax} \left( \frac{X W_Q (X W_K)^T}{\sqrt{d_k}} \right) X W_V$$

# One bank, Two Contextual Meanings

## SENTENCE 1

I deposited money in the bank.

bank → money, deposited

H\_bank (financial)

## SENTENCE 2

I walked along the bank of the river.

bank → river, walked

H\_bank (river)

*E\_bank is identical in both sentences. But  $H\_bank(\text{financial}) \neq H\_bank(\text{river})$  — because attention incorporates different contextual information.*

# Causal / Masked Self-Attention

The patient developed severe fever.

When predicting severe, the model must not look at fever — otherwise it would cheat.

|           | The | patient | developed | severe | fever |
|-----------|-----|---------|-----------|--------|-------|
| The       | ✓   | ✗       | ✗         | ✗      | ✗     |
| patient   | ✓   | ✓       | ✗         | ✗      | ✗     |
| developed | ✓   | ✓       | ✓         | ✗      | ✗     |
| severe    | ✓   | ✓       | ✓         | ✓      | ✗     |
| fever     | ✓   | ✓       | ✓         | ✓      | ✓     |

## CAUSAL MASKING

*also called look-ahead masking*

**$x_t$  cannot attend to  $x_{(t+1)}$ ,  $x_{(t+2)}$ , ...**

This is the type of attention used by GPT-style, autoregressive language models. It guarantees that generation only ever depends on tokens already produced.

***This becomes essential in the next lecture on Language Modeling.***

# Why Have Only One Attention Mechanism?

**The bank approved the loan because the customer had sufficient funds.**

This one sentence contains many different relationships at once:

bank ← approved

bank ← loan

customer ← had

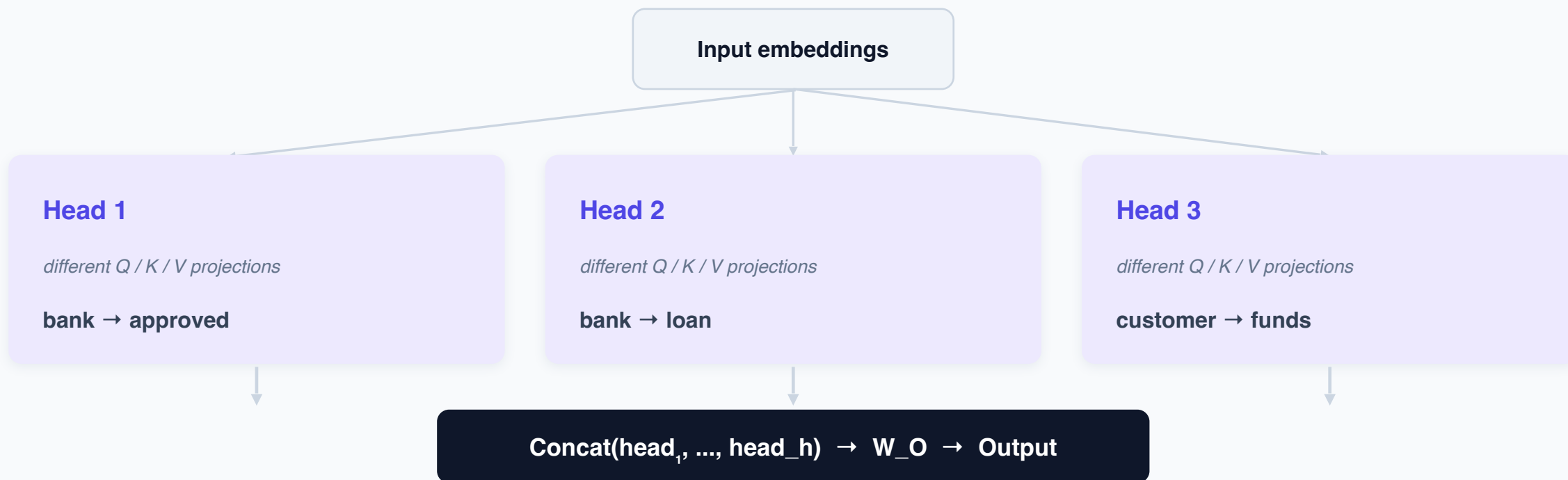
funds ← sufficient

loan ← approved

A single attention mechanism may not be able to represent all of these useful relationships at once.

**So we use multiple attention heads.**

# Multi-Head Attention



$$\text{head}_i = \text{Attention}(Q W_{Qi}, K W_{Ki}, V W_{Vi})$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

*Real heads don't necessarily map cleanly to one human-defined relationship — these are conceptual illustrations.*

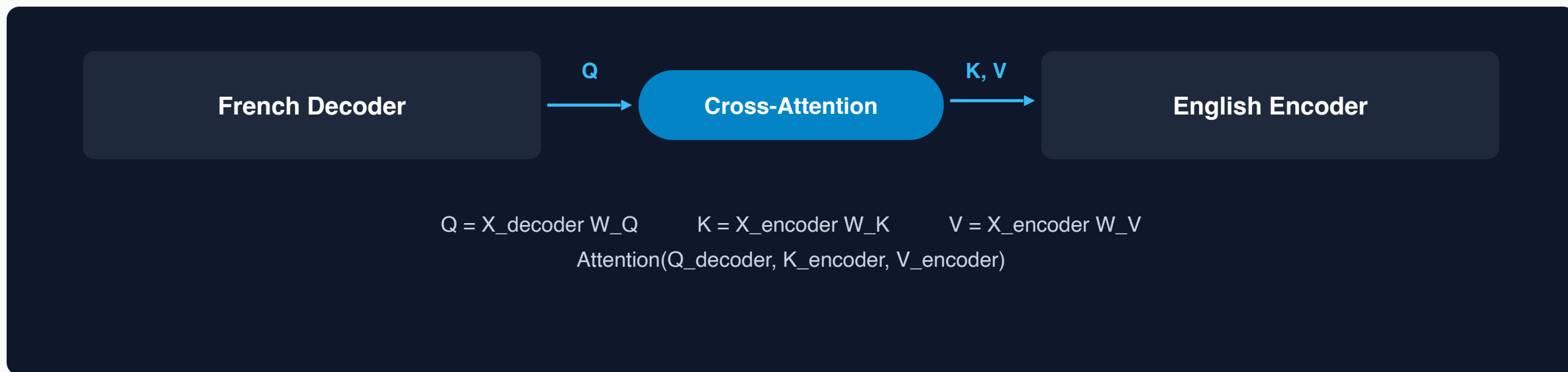
# Self-Attention vs Multi-Head Attention

|                | Self-Attention             | Multi-Head Attention                             |
|----------------|----------------------------|--------------------------------------------------|
| Describes      | Where Q, K, V come from    | How many attention projections run in parallel   |
| Q, K, V source | Same sequence (X)          | Same sequence, split across h heads              |
| Answers        | “What should I attend to?” | “How many different ways can I attend at once?”  |
| Combined as    | —                          | Multi-head self-attention (used in Transformers) |

A Transformer layer uses multi-head self-attention: Q/K/V drawn from the same sequence, projected in parallel across several heads.

# Cross-Attention: Connecting Two Sequences

English: “**The bank is closed.**”    French decoder: “**La banque...**”

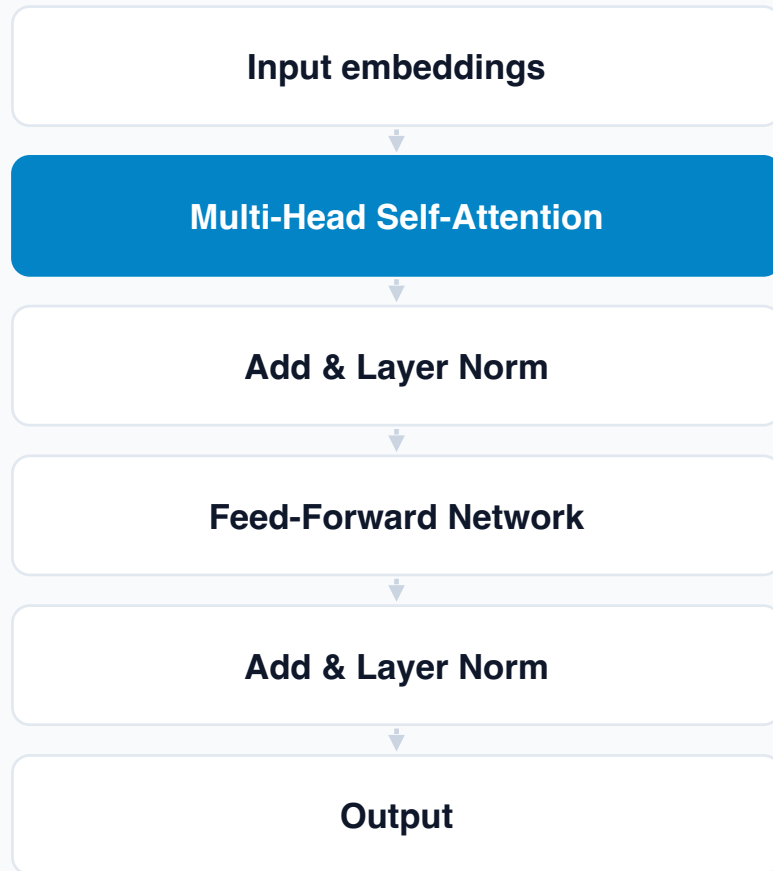


*The decoder asks: which parts of the English input are relevant to what I'm generating right now?*

# Self-Attention vs Cross-Attention

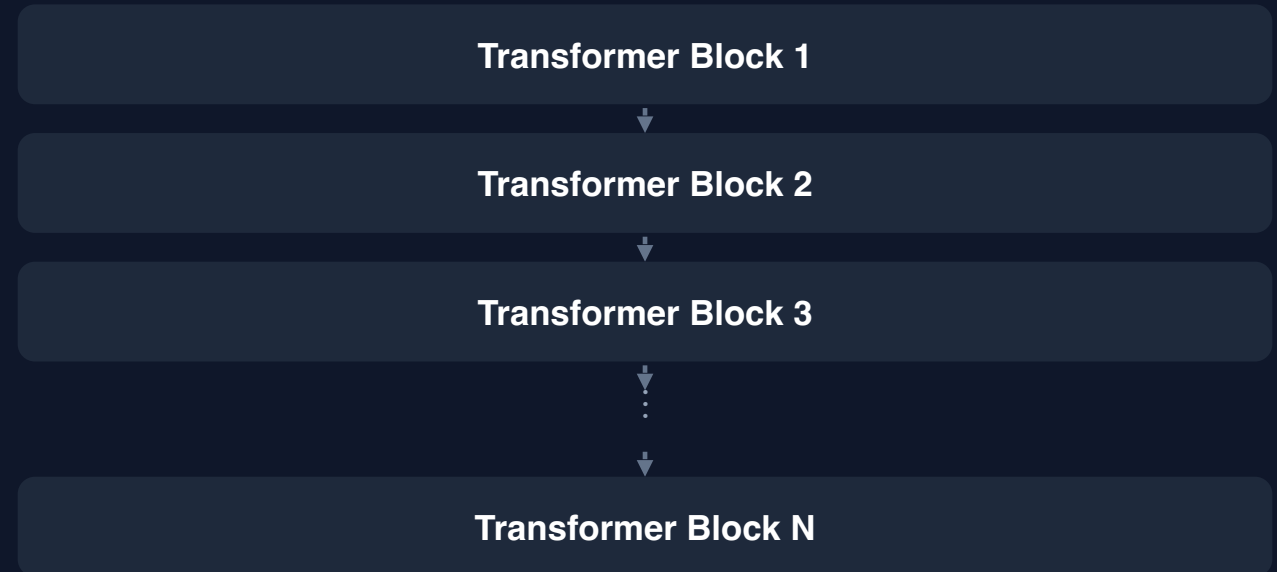
|         | Self-Attention                    | Cross-Attention                  |
|---------|-----------------------------------|----------------------------------|
| Q       | Same sequence                     | Sequence A (e.g. decoder)        |
| K       | Same sequence                     | Sequence B (e.g. encoder)        |
| V       | Same sequence                     | Sequence B (e.g. encoder)        |
| Purpose | Understand internal relationships | Connect two representations      |
| Example | bank ↔ river                      | French decoder ↔ English encoder |

# Attention Inside a Transformer Block



*Repeated N times*

## EMBEDDING



*Each block reuses the same attention mechanism you just learned — stacked and combined with a feed-forward network.*

# What Does Attention Actually Give Us?

## BEFORE ATTENTION

**bank**

Initial token representation — same everywhere, independent of context.

## AFTER ATTENTION

**bank**

Context-aware representation — shaped by whichever tokens were relevant.

**Attention = mechanism for contextualizing representations**

## ATTENTION ≠ MEANING

Attention determines how information from other positions contributes to the current representation. Meaning is distributed across the entire network — attention is one mechanism used to build it, not the meaning itself.

# Limitations of Attention

## COMPUTATIONAL COST

### $O(n^2)$ pairwise interactions

1,000 tokens

~1 million pairs

10,000 tokens

~100 million pairs

100,000 tokens

~10 billion pairs

*One reason long-context inference is expensive.*

## NOT AUTOMATIC EXPLAINABILITY

### High weight $\neq$ causal reason

A high attention weight doesn't necessarily mean “this word is the reason the model made this decision.”

***Attention weights should not automatically be interpreted as causal explanations.***

Attention isn't magic — it's a learned, differentiable weighting scheme with real computational and interpretive limits.

# Lecture 8 Key Takeaways

## 1 The Attention Equation

$\text{softmax}(QK^T / \sqrt{d_k}) V$  turns similarity into a weighted blend of Values.

## 2 Q / K / V Roles

Query asks, Key offers, Value delivers — three projections of the same embedding.

## 3 Causal & Multi-Head

Masking enables autoregressive generation; multiple heads capture parallel relationships.

## 4 Foundation for Transformers

Attention is the core mechanism stacked, with feed-forward layers, into every Transformer block.

# Next Lecture: The Transformer Architecture

## LECTURE 9 PREVIEW

### The Transformer Architecture

We've built attention from first principles. Next, we assemble it with feed-forward networks, residuals, and normalization into the full Transformer block — and see how it enables next-token prediction and generation.

Embeddings



**Attention**



Transformer



Language Modeling



Generation